ChatGPT

Traditional Databases vs Flat Text Files for Long-Term Medical Records

Keeping medical records usable for decades raises unique challenges. Below we compare traditional database storage and flat file text-based storage for electronic health records (EHRs), focusing on longevity and interoperability rather than raw performance. We examine how each approach affects long-term data integrity, data exchange between systems, and the risk of formats becoming obsolete. Real-world examples and expert perspectives (from academia, healthcare IT, and archivists) are included to illustrate the arguments on both sides.

Data Integrity and Consistency Over Decades

Traditional databases (e.g. relational DBMS) are designed with ACID properties to maintain internal consistency. They enforce data types and relationships (foreign keys, constraints), helping prevent inconsistencies during daily operations [1] [2]. This means a patient record split across tables will stay in sync – if an item is updated, it's updated everywhere, preserving coherence of the data over time [2]. Transactional databases also guard against partial writes or concurrent edit conflicts, reducing corruption risk in the short term. Over long timescales, however, keeping a database platform healthy requires ongoing maintenance (software updates, migrations to new versions) so that the data doesn't become trapped in an outdated system. Ensuring *decades-long* integrity involves regular backups, media refresh (to prevent bit rot), and possibly exporting data out of the original database for safekeeping [3] [4]. The U.S. Veterans Health Administration's VistA system is an example: it has reliably stored electronic records in a database (a MUMPS-based system) for decades, but now faces the challenge of archiving or migrating those vast legacy datasets as technology evolves [4].

Flat file text systems (e.g. storing records as CSV, JSON, XML, or HL7 files) lack built-in relational integrity checks. Each file (or each line within a file) is typically a self-contained record, so cross-record constraints must be handled manually or by convention [5] [6]. This simplicity can be double-edged: it's *easier to create and read* plain text files (no complex schema or DB engine needed) [7] [8], but errors like duplicate or inconsistent entries can creep in if not carefully controlled. Over decades, a text record will **remain readable** as long as the character encoding is known – even a simple CSV from 30 years ago can be opened today, reflecting the format's "continued usefulness" and stability [9]. Ensuring integrity in flat files often involves additional measures like checksums or digital signatures. For example, research on long-term EHR authenticity suggests using cryptographic signatures and periodic re-signing to ensure records haven't been tampered with as years pass [10]. Once a flat file is written and finalized (made read-only), it is actually quite robust against silent corruption – any change would be detectable via hashes or the signature scheme. In practice, healthcare organizations using flat file exports for archive often treat them as append-only or read-only logs, avoiding edits that could introduce inconsistencies. The trade-off is that without a live database's integrity enforcement, discipline is required in how data is appended and validated.

Maintaining integrity long-term ultimately requires good governance regardless of format. Database-backed systems need policies for data retention and migration to newer platforms (to avoid the data *staying* correct but becoming *inaccessible*), while flat file archives need metadata and validation to ensure each record remains authentic and traceable to its origin. HL7's Records Management guidelines emphasize storing metadata (timestamps, authorship, etc.) with each record to prove its authenticity

and context decades later [11] [12]. This metadata can be stored in a database or alongside flat files. In short, databases offer strong integrity controls during active use, whereas flat files rely on simplicity and external validation for long-term correctness. Organizations may even use a hybrid approach: active data in a database, and periodic snapshots or exports to flat files for immutable long-term archive (ensuring both integrity and future readability).

Interoperability and Data Exchange

A major consideration for longevity is how easily data can move *between* systems over the years. Healthcare data often outlives the software that created it [13] [14], so interoperability is critical.

Traditional databases often use proprietary schemas or formats that make exchanging data between different EHR systems difficult. One hospital's patient database may not map directly to another's without complex migration scripts or interfaces. Indeed, 64% of healthcare organizations report struggles in transferring legacy data into new systems due to non-standard or proprietary formats [15]. When switching EHR vendors (a common occurrence in healthcare [16]), teams must perform data mapping and conversion. This process can be error-prone and expensive, sometimes requiring third-party tools or consultants to extract data from an old database and import it into the new system [17]. Databases can certainly export data (via CSV extracts, SQL dumps, etc.), but if the original schema is not widely understood, those exports may lose context or require custom parsing.

By contrast, **flat text file systems** can be designed around **standard formats** that maximize interoperability. For example, HL7 v2 messages (plain text delimited), HL7 CDA documents (XML), or FHIR resources (JSON/XML) are all text-based formats expressly created for health data exchange. If long-term records are stored in one of these standard formats, different systems and vendors decades apart can parse and understand the data. An illustrative success is Finland's national health archive (KanTa), which stores medical documents in *HL7 CDA R2 XML* format. This highly structured, standardized text format provides "a solid infrastructure for gathering nationwide health information in the centralized archive" and ensures semantic compatibility across all hospitals in the country [18] [19]. Because every record in that archive follows the same HL7 standard, systems built years later can retrieve and interpret clinical notes without needing the original software that wrote them. Another modern example is the adoption of **HL7 FHIR** for archival: some archiving solutions now convert legacy records into FHIR JSON files so that future systems (which are likely to speak FHIR) can consume them directly [20] [21]. By storing data in an agreed-upon interchange format from the outset, flat file archives can act as a "lingua franca" between old and new EHR systems.

Interoperability is not just about formats but also **accessibility**. A plain text file can be opened with any text editor or processed with a simple script – there's no specialized database engine required. One IT professional noted that unless a DB is *truly* necessary for performance, they prefer a flat file because it's not an "incomprehensible blob" – it can be opened with other tools, parsed, or even put under version control, unlike a proprietary database file [22]. This sentiment highlights that text data can be more readily exchanged and understood. Flat files also make *bulk data transfer* easier: for instance, exporting an entire patient database as a set of CSV or JSON files allows handing off those files to another provider or to researchers, who can import them into whatever system they choose. In fact, research data collaborations often rely on simple flat files (CSV extracts from EHRs) as the lowest-common-denominator format for sharing clinical datasets across institutions [23].

That said, **databases can be interoperable too**, but usually via additional layers. Many EHR systems provide APIs or use standards like ODBC/SQL or FHIR interfaces on top of their database. The data still resides in a database, but it's made exchangeable by exporting to standard formats. The key difference

is that with a flat file approach, the *stored format is itself the standard*, whereas with a database, you typically need an export/transform step. A potential interoperability pitfall of databases is vendor lock-in: if the vendor does not provide good export tools, the data might effectively be stuck. This is why standards bodies and healthcare regulators encourage using open data standards. For example, the U.S. ONC (Office of the National Coordinator) has promoted the use of FHIR and CCD (Consolidated Clinical Document) exports for data portability when patients or providers move between systems [24]. In EHR-to-EHR transitions, common strategies include generating standardized summary files (like CCDs in XML) or using a **3rd-party archive** that converts the proprietary database contents into a neutral format for long-term retention [24].

In summary, flat text formats can greatly simplify interoperability by making the data self-describing and tool-agnostic. Traditional databases usually require more work to bridge between different platforms, though they can support interoperability through standards-compliant APIs or export routines. Over a multi-decade timeline in healthcare, it's often the systems using **standardized, text-based record formats** that have succeeded in seamlessly exchanging or migrating data, while those with closed, proprietary databases have needed extra interventions to avoid data silos.

Risks of Proprietary Formats and Obsolescence

Medical records may need to be retained for decades – sometimes well beyond the lifespan of any given software or storage technology. This raises concern that the format used to store data could become unreadable or unsupported in the future. Here, the choice between a proprietary database and a plain text format has significant implications.

Proprietary database formats (or even proprietary versions of standard databases) carry a risk of obsolescence. If an EHR stores its records in a closed-source or uncommon database, what happens in 20+ years if that database is no longer maintained? The healthcare industry has seen many systems come and go, and data trapped in an obsolete database can become a nightmare. For example, some legacy hospital systems from the 1980s and 1990s stored data in formats or platforms that now require specialized expertise (or old hardware) to retrieve [17]. Migrating from such a system often means hiring experts to extract data or paying the original vendor (if they still exist) to unlock the data. A HIMSS report noted that nearly two-thirds of healthcare organizations struggle with moving legacy data to modern systems due to outdated formats [15]. The problem isn't just theoretical: in practice, hospitals merging or upgrading EHRs frequently encounter this issue. One analysis of EHR transitions observed that data migration is a major challenge, and in some cases the legacy system has to be kept running in read-only mode just so old records remain accessible [25] [24]. This "read-only legacy EHR" approach is essentially a band-aid to cope with proprietary data that wasn't easy to convert.

Proprietary relational databases (like a specific vendor's data files) or niche hierarchical databases can also become **unsupported** as OS and hardware change. If no modern system can directly run the old DB, the data must be exported. This is why digital archivists recommend not relying solely on proprietary binary dumps. The Digital Preservation Coalition advises that there is *"no single perfect file format"* for preserving databases long-term, but the safest route is to capture a complete set of the data in a vendor-neutral form [3]. In practice, archivists often export database contents to **simple text-based formats based on open standards**, keeping the data **vendor-neutral, transparent, and accessible** in the future [3]. Even the Library of Congress recommends formats like CSV or **SQLite** (an open-source, self-contained database) for preserving datasets, since these are widely supported and not tied to a single vendor [3]. The emphasis is on avoiding opaque formats. A running joke in tech is that plain ASCII text from the 1960s is more likely to be retrievable today than data on a 1990s proprietary disk – precisely because text is standard and self-documenting.

Flat text files (especially those using open standards) are relatively immune to format obsolescence. ASCII/UTF-8 text has been readable for decades and will likely remain so. As one engineer noted, the ubiquity of plaintext means "everything today uses and understands" it, and we have many tools (even basic ones like grep or diff) to work with it [26]. A CSV or JSON file created now will very probably be interpretable in 20, 30, 50 years – worst case, one might need to write a small script to parse it, but the format itself is documented and simple. This durability is one reason flat files (like CSV) "have seen fairly little change over the years" and remain useful [9]. Using an **open standard text format** for medical records (e.g. CDA, JSON FHIR, or even plain CSV for certain data extracts) greatly reduces the risk that the format becomes a barrier. Even if specific standards evolve, they are usually backwards compatible or at least openly documented. For example, HL7 CDA has a defined XML schema; even if HL7 were to disappear, any XML parser can still read those files, and the medical concepts can be deciphered from the schema documentation. In contrast, if a proprietary EHR database from 2005 uses a binary encoding and the vendor is gone, one might have to reverse-engineer the file format to extract patient data – a potentially Herculean task.

Another obsolescence concern is **media and hardware**, but modern storage (discussed more below) and migration practices mitigate that. Still, format choice plays into hardware longevity: text files can be migrated from one storage medium to another (say from on-premise servers to cloud object storage, or from HDD to SSD to whatever comes next) without compatibility issues. A database might require ensuring the new platform can run the DB engine or at least host the data files. Proprietary formats can also complicate using new storage paradigms – for instance, a specialized older database might not be directly supported in a cloud storage service, whereas flat files can be stored virtually anywhere.

In summary, relying on a **closed or proprietary database format introduces long-term risk** that the format itself could become unreadable. History has shown that healthcare providers often must undertake special projects to rescue or convert data from retired systems [14]. On the other hand, using **open, text-based formats** for record storage emphasizes longevity – future systems, even if not invented yet, will almost certainly be able to parse plain text. This is why information archivists tend to favor flat files or standardized data packets for preservation [3], and why forward-looking healthcare IT initiatives (like openEHR) focus on keeping data independent of any single application or vendor [27] [14].

Real-World Examples: Successes and Struggles

Success with a traditional database: The U.S. Veterans Affairs' **VistA** EHR system is often cited as a success in terms of longevity. It has used an integrated database (based on the MUMPS language and later InterSystems Caché/IRIS) for nearly *40 years*, retaining millions of veterans' records. VistA's database is not a standard relational model – it's a hierarchical key-value store – but it was open-source and well-documented. This openness meant the VA could maintain and update the system internally for decades. However, as the VA plans to transition to a new system, even this "successful" long-lived database presents a challenge: how to **archive or migrate** all that historical electronic data. The VA has acknowledged an "archiving problem for digital data" and begun addressing how to preserve or transfer those records [4]. The takeaway is that a database can definitely hold up over decades *if actively maintained*, but eventually one faces the question of moving to newer technology. VistA's data is being migrated to a commercial EHR now, and the effort underlines the difficulty of mapping a bespoke schema to a new one (a process that requires extensive validation to ensure no data integrity is lost during conversion).

Another example is **openEHR-based systems** in places like Norway and Brazil, which use a vendor-neutral data repository (often implemented with a relational or document database underneath, but

structured by openEHR archetypes). These have seen success in ensuring that data remains usable even if front-end applications change. openEHR's philosophy is that the data is stored in a standardized format (archetypes/JSON) independent of any single vendor application, so it can survive multiple generations of software. As an openEHR advocate describes, *"What is recorded today will be data that … can still [be used] in 10 or 100 years"* because the data isn't locked to a proprietary model [27]. One could categorize this as a hybrid approach: using a **modern database engine** for performance, but the data format itself is an open, self-describing text-based model. This has succeeded in some large regional health systems where they've replaced aging hospital IT with an openEHR platform – they report easier data sharing and future-proofing, since adding new apps or analytics doesn't require reshaping the legacy data [14].

Success with flat file approaches: A clear success story is the Finnish **KanTa** national health archive. Since 2007, Finland mandated that all healthcare providers send standardized clinical documents to a central archive built on HL7 CDA (an XML flat file format) [28] [19]. This means that when a patient in Finland has an encounter, the summary or note is saved locally *and* sent as a CDA document to the archive. The archive effectively stores millions of XML files. Because CDA is highly structured and standardized, the archive can be queried and new systems can retrieve old documents without needing the original EHR software. This flat file repository approach has **succeeded** in providing nationwide interoperability and long-term retention. Over a decade in, multiple different EHR products in Finland all connect to the KanTa archive, demonstrating that a text-based, standardized storage can be scaled and maintained. The trade-off is that CDA documents are document-centric (suitable for reading a whole report) and not a live database you can query for, say, "list of all lab results in 2010" without first parsing the documents. Finland addressed this by ensuring that while day-to-day operations use databases at each hospital, the archive (for longevity) is the centralized CDA store. It shows a practical model: **active data in databases, long-term copy in standardized flat files**.

Another example is how some hospitals handle **legacy EHR retirement**. When retiring an old system, a common successful strategy is to **export all patient records to PDF or text** and store those in an archive system. For instance, one health IT forum discussion described exporting records as PDF "flat files" which could then be accessed as needed in the future without the original software [29]. PDF is not plain text, but it is a widely readable format (an open ISO standard) suitable for long-term documents. Some organizations export structured data as CSV for things like patient lists or as **CCD** (an XML summary) for each patient, and scan unstructured notes to PDF. These approaches have succeeded in the sense that years later, the organization can still pull up an old record – essentially treating it as an electronic paper chart. The downside is that this loses the rich structuring of a live database (you can't easily run analytics on a bunch of PDFs), but it meets the longevity and legal retention requirements. Many healthcare providers have used third-party archiving vendors who specialize in taking an EHR database and turning it into a collection of flat files plus a simple viewer. The success here is measured by **accessibility**: e.g., a hospital that migrated to Epic still keeps a legacy data archive where a clinician can view older records via PDF or XML, without needing to maintain the legacy database server. This has worked well to ensure compliance with retention laws and to have information available in a pinch (like an old operative report from 15 years ago).

Struggles and failures: On the other side, there are cautionary tales. Some hospitals that failed to plan for data export found themselves effectively **locked out** of their own legacy data. For example, if a small practice used a PC-based proprietary database for its EMR and the vendor went out of business, the practice might have had to hire a specialist to extract the data or risk losing it. There have been instances where data was eventually retrieved but only after significant effort, or where only a portion could be salvaged. In academia, a 2020 review noted that there is "scant literature" on EHR-to-EHR transitions, and one of the biggest issues is *data migration*, including ensuring data integrity and semantic interoperability when moving between different systems [16]. This implies many institutions

have struggled, often opting to keep an old system running read-only (which is costly and not scalable long-term).

Another struggle is when **data standards evolve** – for instance, an early adopter of HL7 v3 CDA might later want to transition to FHIR JSON. If their data is in flat files but an older standard, some translation is needed (though at least both are open formats). By contrast, if data was in a proprietary schema, one has to first get it out, then transform. A minor but practical issue with flat files can be **sheer volume**: decades of records in text form might be billions of lines or files. Without an indexing mechanism, retrieving specific information could be slow. Some systems that went naive on flat files found performance suffered as data grew (one Stack Overflow user noted a tool became "hamstrung" by using XML files without a database once the data scaled up [30]). This is more of a performance concern than an interoperability one, but it speaks to why pure flat file systems fell out of favor for operational use. Modern solutions mitigate this by using search indexes on top of flat files (e.g. an archive might store JSON files but index them with Elasticsearch for quick retrieval).

In summary, many **successful strategies use a blend**: leveraging databases for day-to-day use and flat files or open formats for long-term archival and interchange. Failures tend to occur when data is kept in a closed box until that box breaks or becomes obsolete. Whether using databases or flat files, the key is planning for eventual export, migration, or reading by systems that didn't create the data. As one health IT professional wryly observed about legacy data, "not throwing anything away" is common but the value is lost if you can't *use* the old data in new contexts [31] [32].

Impact of Modern Storage Technologies (SSDs, Cloud, etc.)

Modern storage technologies have changed the landscape in terms of capacity, reliability, and how data can be managed, but they don't completely eliminate the fundamental trade-offs between databases and flat files for longevity.

- **Solid-State Drives (SSDs)**: SSDs offer fast access and have no moving parts, reducing some traditional concerns about data retrieval speed from flat files. Decades ago, one might worry that scanning a huge text file on a slow disk would be untenable; with SSDs (and large RAM caches), even quite large text files can be searched relatively quickly. This diminishes the performance advantage databases held in certain read scenarios (one commentator noted that if you were using a database primarily to reduce disk I/O, "it may not be necessary with fast SSDs" in play [33]). However, SSDs have their own longevity concerns – they can lose data if left unpowered for long periods, though in practice enterprise systems continually refresh the charge. From a long-term integrity perspective, SSDs are typically used in data centers with redundancy and error correction, so whether the data is in a flat file or a database file doesn't matter much; the storage system will keep multiple copies or use RAID. The speed of SSDs can even make a flat-file approach more feasible, since the hardware can handle ad-hoc parsing of text when needed.

- **Cloud storage**: Cloud services have introduced highly durable storage solutions (e.g. object storage like Amazon S3 guarantees 99.999999999% durability by replicating data across locations). This greatly benefits long-term record keeping. Notably, cloud object storage is *schema-agnostic* – it stores files, not structured tables. This makes it naturally aligned with a flat file paradigm. Healthcare providers can archive text files (CSV, JSON, XML, PDF, etc.) to cloud storage and be confident the bits will persist and be reachable from future systems. Cloud vendors also offer managed database services, which ensure your database software is kept up to date and hardware issues are abstracted away. This solves the *maintenance* aspect of longevity

to some extent: for example, using a cloud-managed PostgreSQL means you're less likely to end up with an un-run-able old version. But it does introduce *vendor dependence*. If in 30 years you want to move away from that cloud, you'd still need to export the data in a usable form. So the caution about proprietary formats extends to cloud-provided databases too – you must ensure you can get your data out. Cloud environments somewhat blur the lines between flat file and database: for instance, you can store flat files in the cloud and use on-demand query services to analyze them (treating them like a database when needed). This means the old performance rationale for databases is less pressing for archival data. One can keep the archive in textual form and use tools like BigQuery or AWS Athena to run SQL-like queries on those files without ever loading them into a traditional DB.

- **Modern databases and noSQL**: Newer database technologies (NoSQL document stores, graph DBs, etc.) often store data in semi-structured forms (JSON documents, for example). In some cases, these blur the distinction – a document database might internally use JSON (which is text-based) but compressed in a binary. The key for longevity is whether the data can be extracted in a standard form. Modern systems like MongoDB or CouchDB (JSON document stores) at least make it straightforward to export JSON, which is a good practice for interoperability. Also, some modern SQL databases (PostgreSQL, etc.) can store JSON columns, effectively allowing a hybrid of relational and flat data. These advances mean that even if you choose a "database" approach, you might still decide to store the critical clinical data in a JSON column following a standard schema (like FHIR resource schema), thus gaining the benefits of both worlds: ACID storage and an open format within it.

- **Computing power**: With much faster CPUs and distributed computing, one argument *against* flat files (difficulty of parsing large files or doing complex analysis) is mitigated. Even if you have decades of records in text, modern processors or cloud compute clusters can process them when needed. This doesn't directly impact longevity, but it means choosing flat files isn't as limiting as it might have been in the past when only a powerful RDBMS could effectively query the data. We also have better indexing and search technology (e.g. indexing text with Apache Lucene/Elasticsearch) which can be layered on top of files.

- **Backup and replication**: Modern storage makes it easy to keep multiple copies of data (on-site, off-site, cloud). This is crucial for longevity: no matter how data is stored, it should be replicated and periodically checked. Traditional databases often use transaction logs and backups for this; flat files might use version control or periodic snapshots. With cloud, one can automate snapshots of a database or versioning of files. Importantly, cloud storage can alleviate the need to migrate physical media every few years (the cloud provider will handle migrating to new disks over time). This means whether you have a 20-year-old flat file or a 20-year-old database, if it's been sitting in well-managed cloud storage, the bits are probably intact. The open question will be format readability, which brings us back to using standard or simple formats for long-term compatibility.

In essence, **modern hardware and cloud services reduce the operational differences** between using a flat file or a database for archival. Performance and storage capacity are less of a concern, freeing us to prioritize longevity and interoperability. This aligns with expert recommendations: choose the solution that minimizes future dependencies. For many, that means leaning toward simpler, self-describing formats (which flat files excel at) unless there's a compelling reason to rely on a complex database structure. As one commentator put it, at the hardware level "it is all bytes on a disk" anyway [34] – modern systems let us choose how much of the organizational burden we put on the application vs. the storage layer. With abundant RAM and fast SSDs, some tasks that used to *require* a database can be handled with files. Still, for active clinical use, databases remain invaluable for ensuring consistency

and supporting multi-user transactions – modern tech hasn't obviated that. But for *archival and interchange*, the advances in storage mean the simplicity of flat files can be embraced without as heavy a penalty.

Expert Perspectives and Best Practices

Academia & research: Academic researchers in health informatics have long pointed out the importance of data longevity and standardization. In a 2007 paper on long-term EHR authenticity, researchers highlighted that the lifespan of healthcare data will exceed the lifespan of any single digital signature or encryption key, implying that records need periodic maintenance to remain verifiably authentic [10]. This kind of work suggests that **format aside**, healthcare data needs active curation over time – whether you use a DB or flat files, you must plan for things like re-signing data, migrating to new storage media, and updating formats as needed to avoid bit rot and loss of trust. Another scholarly discussion by Scott et al. (2006) noted the lack of published work on general medical record preservation and emphasized that preservation policies must "take advantage of IT for facilitating access" to archived data [31] [32]. In other words, simply keeping data is not enough; it must be kept in a way that future IT systems can easily access and use it. This implicitly argues for **interoperable, open formats** and against black-box storage.

Academic and industry standards groups have converged on promoting **open standards** like HL7 CDA and FHIR for ensuring that even if databases differ, the *data* can be exchanged. Standards organizations (HL7, ISO, openEHR) have published models for what an archival record should contain. For instance, HL7's EHR Functional Model includes requirements for **metadata** (provenance information: who wrote a note, when, etc.) to travel with the record for legal integrity [11] [12]. They don't mandate the storage format but emphasize the content and context that must persist. Many academic papers in recent years also explore using FHIR as not just an exchange format but a storage format. The rationale is that if your data is already in FHIR JSON, it's immediately ready to be shared or understood by any other FHIR-compliant system [20] [35]. This echoes the research community's push toward **self-describing data** and aligns with broader data science trends (where researchers prefer CSV/JSON because they can load it into any analysis tool).

Healthcare IT professionals: Folks on the ground in hospitals and clinics often stress pragmatism. They value solutions that meet regulatory requirements (e.g. retention laws) and that don't disrupt clinical workflows. There's a recognition that keeping legacy systems running just for data access is costly, so archiving to a simpler format is appealing. Many health IT professionals advocate for extracting data from old EHRs into an **archive that clinicians can query** (even if just a read-only web portal that pulls from archived files) [36] [37]. Professionals often cite the need for "continuity of care" during transitions – meaning the new system must have *some* access to the old data, whether via database conversion or a flat file archive [36]. The consensus in forums like /r/healthIT and at HIMSS conferences is that **data archival is an essential part of EHR lifecycle**. A best practice is to convert legacy data into a non-proprietary format and store it in a secure archive *before* decommissioning the old system [17]. For example, one might extract all patient demographics and key clinical data into CSV or XML files and use an archival viewer for lookup. Healthcare IT staff also warn about **vendor lock-in** from experience – many have struggled with vendors who charge hefty fees to output data in usable form when switching systems [17]. This creates a strong argument in the community for **owning your data in an accessible format**. That often translates to pushing vendors to support standards (like "please provide us a CCD for each patient on exit") or choosing systems that use mainstream databases where at least you can get direct SQL access to your tables.

Information archivists: Archivists and digital preservation experts bring a long-term viewpoint that complements the immediate needs of clinicians. Archivists emphasize format sustainability, documentation, and independence from specific software. As noted, guidelines from preservation organizations recommend using open, text-based formats for longevity [3]. Archivists also look at things like **format migration** schedules – planning that every X years, data will be checked and possibly converted to newer formats if the current one shows signs of becoming obsolete. In healthcare, archivists might work alongside IT to ensure that, for instance, an old proprietary format is converted to an XML standard *before* the only machine that can read it dies. They also stress maintaining **data dictionaries and schema descriptions** so that even if the data is in CSV, you know what each column means 30 years later. One archivist perspective is that relational databases are great now, but if you don't also preserve the schema and relationships in a human-readable way, you could lose the meaning of the data down the road [38] [39]. A strategy archivists use is to export not just the data tables but also the schema (table definitions, relationships) and possibly a small sample database in an open format like SIARD (Software Independent Archiving of Relational Databases) [40] [41]. SIARD essentially wraps the database schema and content in a documented XML-based format. This is a way to have the best of both worlds: you capture the full fidelity of the relational data (all tables and links) but store it in an open, long-term form.

In summary, expert opinions tend to converge on a few key **best practices** for long-term medical record storage:

- **Use open or standard formats whenever possible** – This ensures future interoperability and reduces risk of obsolescence [3] [20]. If using a database, choose one with easy export to standard formats or consider an open-source engine like PostgreSQL or SQLite which are unlikely to become unreadable [3].

- **Maintain metadata and context** – Know that decades later, you'll need to understand who wrote a record, when, and under what context. Store that metadata alongside the data. This is crucial for legal defensibility and clinical understanding of old records [11] [12].

- **Plan for transitions** – Assume that you will change systems or storage tech at some point. Incorporate data export and archival into EHR replacement projects, rather than an afterthought [16] [24]. Many recommend using **archival platforms** as intermediaries during migrations: legacy data is exported (often as flat files or PDFs) into an archive, and the new EHR can link to that archive for historical lookup [24].

- **Balance structured and unstructured data** – Not everything needs to be a perfectly normalized database for 50+ years. For data needed in computations (e.g. lab results for longitudinal studies), keeping a structured form (maybe a CSV or a small database) is useful. For narrative or document-like data (old consult notes, discharge summaries), PDF or CDA documents might suffice. A combination can ensure both human readability and data-mining capability.

- **Security and integrity** – Apply security measures to whichever storage you choose. Flat files should be access-controlled just like a database would be, and integrity checks (hashes, digital signatures) should be used so you can detect any corruption or tampering over time [10]. Databases similarly require security patches and audit logs to maintain trust in the data.

- **Periodically re-evaluate** – Every so often, assess if the format you are using is still widely supported. If not, convert the archive to a newer standard. For instance, if an organization had

archives in HL7 v2 (pipe-delimited text) from the 1990s, they might now convert those to FHIR JSON to make them more accessible to modern tools – because FHIR has libraries in many languages, while HL7 v2 parsing is becoming a niche skill. The earlier point from openEHR is relevant: design the data store to be "evergreen" – adaptable to new technologies without losing the core information [14].

In conclusion, when comparing traditional databases to flat file text systems for long-term medical record storage, it's clear there is no one-size-fits-all answer. **Traditional databases** offer strong immediate data integrity and efficient multi-user access, which is essential during clinical operations, but they require vigilance to avoid future format lock-in. **Flat file systems** (especially using standard formats) excel in portability, transparency, and longevity of readability, but need careful management of data consistency and can sacrifice some ease of querying. Modern trends in healthcare IT favor a hybrid approach: leveraging databases for current needs and *also* storing data in an archival, interoperable form. As one commentator aptly noted, the choice often comes down to preferring something "readable" over something proprietary whenever performance doesn't demand otherwise [22]. By following expert best practices – using open standards, preserving context, and planning for change – healthcare organizations can ensure that today's medical records remain intact, accessible, and meaningful for decades to come, no matter how technology evolves.

Sources:

- Flatirons. *What is a Flat File, or Flat File Database?* – Explanation of flat file databases vs relational, noting simplicity, portability, and integrity trade-offs [5] [6] [42].

- Thomson, et al. (Digital Preservation Coalition). *Preserving Databases* – Recommends capturing database content in vendor-neutral, text-based formats for long-term access [3].

- Triyam (2023). *Data Longevity: Securing Patient Records for a Lifetime* – Discusses challenges with legacy data migration; 64% of orgs struggle to transfer legacy EHR data to new systems [15].

- Wilcosky (2022). *Rise, fall, and rise of flat file software* (Hacker News discussion) – Developers emphasize readable plaintext formats for longevity and integration with other tools [22].

- HL7 Finland (Suna, 2011). *Finnish National Archive of Health Information (KanTa)* – Describes Finland's centralized archive using HL7 CDA text documents for nationwide interoperability [28] [19].

- 314e (2023). *FHIR-native Healthcare Data Archiving* – Industry perspective on converting legacy EHR data to FHIR JSON for seamless future integration [20] [21].

- OpenEHR Foundation (2023). *The future of digital health depends on it being open* – Argues that open data standards (openEHR) ensure data remains usable for "10 or 100 years," avoiding costly migrations with proprietary systems [27] [14].

- Winkelman et al. (2008). *Archiving the Phenome* (PMCID: PMC2605592) – Highlights the need for long-term preservation of clinical data and notes varying retention requirements (e.g. VA keeps records 75+ years) [4] [43].

- Murphy et al. (2020). *Transitions from One EHR to Another* (PMCID: PMC7657707) – Discusses challenges in EHR data transitions; mentions using consolidated clinical document (CCD) or FHIR for data migration [24].

- Lekkas & Gritzalis (2007). *Long-term verifiability of EHR authenticity* – Suggests using digital signatures and time-stamping to preserve integrity of electronic records over long periods [10].

---

[1] [30] sql - Database vs Flat Text File: What are some technical reasons for choosing one over another when performance isn't an issue? - Stack Overflow
https://stackoverflow.com/questions/1499239/database-vs-flat-text-file-what-are-some-technical-reasons-for-choosing-one-ove

[2] [5] [6] [7] [8] [9] [42] What is a Flat File, or Flat File Database?
https://flatirons.com/blog/what-is-flat-file-or-flat-file-database/

[3] [38] [39] [40] [41] dpconline.org
https://www.dpconline.org/docs/technology-watch-reports/2470-preserving-databases/file

[4] [13] [31] [32] [43] Archiving the Phenome: Clinical Records Deserve Long-term Preservation - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC2605592/

[10] Long-term verifiability of the electronic healthcare records' authenticity - PubMed
https://pubmed.ncbi.nlm.nih.gov/17055778/

[11] [12] Health Level Seven Records Management & Evidentiary Support (RM-ES): Supporting Clinical Documentation for Legal and Billing Purposes
https://www.healthit.gov/sites/default/files/facas/hl7_rmes_hitpc_mu-cawg_cl.pdf

[14] [27] “The future of digital health depends on it being open” – openehr.org
https://openehr.org/the-future-of-digital-health-depends-on-it-being-open/

[15] Data Longevity: Securing Patient Records for a Lifetime of Health
https://www.triyam.com/articles/data-longevity-securing-patient-records-for-a-lifetime-of-health

[16] [24] [25] Transitions from One Electronic Health Record to Another: Challenges, Pitfalls, and Recommendations - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC7657707/

[17] Migrating to a Modern EHR System with Legacy Data Archive ...
https://www.triyam.com/migrating-to-a-modern-ehr-system-with-legacy-data-archive-solutions

[18] [19] [28] Finnish National Archive of Health Information (KanTa): General Concepts and Information Model
https://www.fujitsu.com/sg/imagesgig5/paper15.pdf

[20] [21] [35] [36] [37] Healthcare Data Archival for EHR Migration
https://www.314e.com/muspell-archive/blog/healthcare-data-archival-an-essential-for-ehr-migration-and-data-transformation/

[22] [26] [33] [34] Smaller is better – The rise, fall, and rise of flat file software | Hacker News
https://news.ycombinator.com/item?id=31467969

[23] Data Formats - Rethinking Clinical Trials
https://rethinkingclinicaltrials.org/chapters/conduct/acquiring-real-world-data/data-formats/

[29] EHR Migration and Archiving : r/healthIT - Reddit
https://www.reddit.com/r/healthIT/comments/uo38vp/ehr_migration_and_archiving/